S

SMART STUDENT HUB

BSC INFORMATION SCIENCE

Tamile Ntshalintshali_202304427 – Team leader

Nosipho Shabangu_202301741 – Architect & Designer

Karabo Nkonyane_202302432 – Programmer

Contents

SMART STUDENT HUB: Task Tracking & Deadline Reminder System for UNESWA students

1.INTRODUCTION

The Smart Student Hub is a web-based system that is designed to help students at the University of Eswatini in managing academic responsibilities more effectively. University students often deal with multiple tasks such as assignments, tests which requires proper planning and time management.

Currently students rely more on brain memory, informal reminders and the frequent checking of Moodle. However, these methods are not always reliable because it depends on the students actively remembering or checking for updates. As a results important deadline maybe missed, leading to poor academic performance and increase stress.

The purpose of this project is to develop a simple and user-friendly system that allows students to record tasks and assign deadlines, test dates and receive visual reminders when deadlines are approaching.

1.1Team Members:

- Tamile Ntshalintshali_202304427 – Team leader
- Nosipho_Shabangu_202301741 – Architect & Designer
- Karabo_Nkonyane_202302432 – Programmer

2.USER/CLIENT INVOLVEMENT

The primary or target users are students at the University of Eswatini. Their involment is important throughout the development process to ensure that the system meets their needs.

2.2 User Involvement during Development

- Students will provide information about common challenges they face in managing tasks
- Suggest features they would like in the system
- Test the system and give feedback on the usability and functionality

This is to ensure that the system is built based on real user needs or user-centered design not assumptions

2.3 User Responsibilities

- Enter their academic tasks (tests and assignment, dates and deadlines)
- Update task information when necessary
- Use the system regularly to track deadlines

2.4 Resources provided by users

- Tasks detail (assignment names and deadlines, test dates)
- Feedback on system performance and usability

2.5 Timing of user involvement

Planning stage – provide requirements

Development stage – test early versions

Testing stage – give final feedback

3. Risks

RISK	IMPACT	MITIGATION
Poor internet connection	Difficult in uploading code and collaborating on GitHub	Work offline and upload changes later when internet is available
Lack of teamwork	Lack of communication, delay and incomplete work	Regular meetings
Technical difficulties	Errors in coding and system failure	Conduct Research, testing and seek help when needed
Time Constraints	Project may not be completed on time	Follow a strict schedule and divide tasks properly
Data loss	Loss of project files and the code	Use of version control tools like Git and backups on GitHub

4. STANDARDS, GUIDELINES and PROCEDURES

The project will follow these standards

1. Coding standards (clean, readable and well-structured code
    - Use meaningful variable and function names
    - Comment code where necessary

2. Version Control using Git
    - Use Git to track code changes
    - Upload and manage the project on GitHub
    - Use commits to save progress regularly
    - Use branches for developing new features

3. Documentation standards
    - maintain clear project documentation
    - update README files on GitHub
    - ensure all work is properly recorded

4. Development guidelines
    - Follow step-by-step development process (plan – design – develop – test)
    - Focus on simplicity and functionality
    - Ensure all features meet the requirements
5. Collaboration procedure
    - Each team member is assigned specific tasks
    - Regular communication is maintained
    - Progress is reviewed frequently

By following these standards ensures high quality code, smooth teamwork, easy maintenance of the system

5.PROJECT ORGANISATION

This section of the project clearly explains how the project is structured in relation to other existing student information system. It also explains the role of different team members on how they relate to the different modules that have to be implemented into the Student Hub system.

5.1 Relation of the Student Hub system with other systems

The **Student Hub System** is related to other academic management systems such as:

- Learning Management Systems (LMS)
- Student Information Systems (SIS)
- University Portal Systems
- Online calendar and reminder applications

However, this project is different because:

- It focuses mainly on tracking assignment deadlines and test dates.
- It provides reminder notifications to students.
- It is designed to improve time management and reduce missed deadlines.

However, the system is later expected to integrate with existing university systems to automatically retrieve course information. Therefore, this project complements other academic systems rather than replacing them. It functions with the same aim of promoting the academic life of students.

5.3 Project Structure and Team Organization

The Student Hub System will be developed using a structured project team. Each team member has defined responsibilities to ensure smooth coordination and successful completion of the project.

1. Project Manager

The Project Manager is responsible for the overall management of the project. Project Manager ensures that all phases of the project are completed on time.

Major responsibilities include:

- Planning project activities which should be Specific, Measurable, Attainable, Realistic and Time-bound
- Preparing the project schedule and planning project initiation phase and project completion while adequately allocating resources.
- Allocating tasks to team members
- Monitoring progress which can be done through key performance indicators and assessing for strategic alignment.
- Managing project risks which could involve scope creep and budget shortages.
- Ensuring the project is completed within time and budget

System Designer

The System Designer converts requirements into system structure.

Responsibilities include:

- Designing system architecture
- Designing database structure
- Designing user interface layouts
- Ensuring system usability

The designer ensures that the system is logically structured and easy to use.

Programmer / Developer

The Programmer develops the actual system. The developer also transforms the system design into a working application.

Responsibilities include:

- Writing source code
- Developing front-end and back-end components
- Connecting the system to the database
- Integrating system modules

System Architect

The System Architect is responsible for designing the overall structure of the Student Hub System. While the programmer writes the code, the architect plans how all parts of the system will fit and work together. The task of the system architect differs from the software designer in that the system architect defines the system requirements and what will be required to secure the system. For example, the system architect decides that MYSQL and a client will be required to implement the system. system architect also ensures system is scalable, secure, efficient and reliable by observing and properly planning how different modules of the system will be organized and how they will integrate with each other. While the system designer will be useful in the following means:

- Designs the "Assignments" table with fields (Title, Due Date, Course Code)
- Designs the dashboard layout
- Draws the ER diagram
- Designs how reminders are triggered

5.4 Training Requirements and Schedule

To ensure effective project execution, team members will undergo training in:

- Programming language (e.g., Python, PHP, or JavaScript)
- Database management systems (MySQL)
- Using Git for version control
- UI/UX design tools
- Software testing techniques

Training Schedule

TASK	DESCRIPTION	PREDECCESSOR	WEEK
A	Introduction to project tools and GitHub	None	1
B	Database design and SQL training	A	2
C	Web development framework training	B	2
D	Software testing and debugging tools	C	3

Gantt Chart of training schedule

	WEEK1	WEEK 2	WEEK 3	WEEK 4	WEEK 5	WEEK 6	WEEK7	WEEK8
TASK A								
TASK B								
TASK C								
TASK D								

6. PROJECT PHASES

6.1 Selected Life Cycle Model

The **Waterfall Model** is selected for this project. Each phase must be completed before moving to the next phase. The Student Hub System follows the Waterfall Model, meaning each phase is completed fully before moving to the next. Below is how each Waterfall phase applies directly to your project.

Reasons:

- Requirements are clearly defined.
- The system scope is stable.
- Suitable for academic projects.
- Provides clear documentation at each stage.

The Waterfall Model of the Student Hub System.

1. Requirements Analysis Phase

(Waterfall Phase 1)

In this phase, we define what the Student Hub System must do. It answers what the system should do.

Activities in the Project:

- Interview university students about challenges in tracking deadlines.
- Identify required features:
  - Student registration and login
  - Add assignments
  - Add test/exam dates
  - Calendar view
  - Reminder notifications
- Identify non-functional requirements:
  - Security
  - User-friendliness
  - Fast performance

Output:

✔ Software Requirements Specification (SRS) document
✔ Approved list of system features

2. System Design Phase

(Waterfall Phase 2)

In this phase, we decide how the system will work. It answers the question on how the system should be built.

Activities in Your Project:

- Design system architecture (Client–Server model).
- Create database design:
  - Students table

    - Courses table
    - Assignments table
    - Tests table
    - Notifications table
- Draw diagrams:
    - Use Case Diagram
    - ER Diagram
    - Data Flow Diagram
- Design user interface (login page, dashboard, calendar view).

Output:

✔ System Design Document
✔ Approved diagrams
✔ Database schema

3. Implementation Phase

(Waterfall Phase 3)

In this phase, the actual system is developed. This phase produces the actual software based on the design. Once coding is completed, the system moves to testing.

Activities in the Project:

- Set up development environment.
- Create database tables.
- Develop login and authentication system.
- Develop dashboard interface.
- Develop assignment tracking module.
- Develop test tracking module.
- Develop notification/reminder system.
- Integrate all modules.

Output:

✔ Working Student Hub prototype

4. Testing Phase

(Waterfall Phase 4)

In this phase, the system is tested to ensure it works correctly. This phase ensures the system meets requirements defined in Phase 1.

Activities in the Project:

- Unit testing (test each module separately).
- Integration testing (test combined modules).
- System testing (test complete system).
- User Acceptance Testing (students test the system).

What is Tested?

- Can students log in successfully?
- Are assignments saved correctly?
- Are reminders sent properly?
- Does the system handle errors correctly?

Output:

✔ Tested and verified system
✔ Bug fixes completed

5. Deployment Phase

(Waterfall Phase 5)

In this phase, the system is installed and made available to users.

Activities in the Project:

- Install system on server.
- Configure database.
- Create user accounts.
- Train students on how to use the system.

Output:

✔ System is live and operational

Students can now use the Student Hub System.

6. Maintenance Phase

(Waterfall Phase 6)

After deployment, the system requires maintenance. This phase ensures the system continues functioning effectively.

Activities in the Project:

- Fix bugs reported by users.
- Improve performance.
- Add new features (e.g., exam timetable integration).
- Update security features.

7. RQUIREMENTS ANALYSIS AND DESIGN.

7.1 Requirements Analysis Methods

The following methods will be used:

1. Interviews – To understand student challenges.
2. Questionnaires – To collect data from many students.
3. Observation – To study how students currently manage deadlines.
4. Document Analysis – Reviewing course outlines and academic calendars**.**

7.2 Functional Requirements

The system must:

- Allow students to register and log in.
- Allow students to add assignments.
- Allow students to add test dates.
- Display deadlines in calendar format.
- Send reminders before deadlines.
- Allow students to update or delete tasks.
- Provide notification alerts.

7.3 Non-Functional Requirements

- System must be secure.
- System must be user-friendly.

- System must load quickly.
- System must be reliable.
- System must protect student data.

7.4 Design Techniques

The following design techniques will be used:

- Use Case Diagrams
- Entity Relationship (ER) Diagrams
- Data Flow Diagrams (DFD)
- Prototyping

7.5 System Architecture

The system will use a Client-Server Architecture:

- Front-end: Web interface for students.
- Back-end: Server handling logic.
- Database: Stores assignments, tests, and student data.

7.6 Database Design

Main Tables:

- Students
- Courses
- Assignments
- Tests
- Notifications

Relationships:

- One student can have many assignments.
- One course can have multiple tests.

8.TESTING STAGE

In software development, testing environment is a controlled setup where the Smart -Student – hub system will be run to find bud and verify features before the product goes live to be used by students. The testing will be conducted from the following:

- **Hardware: laptops and smartphones**
    - We will be testing of different devices to ensure responsiveness of the device. It helps to choose the correct device foe the system. A website might look perfect on a wide laptop screen but became unstable on or not work at all in Smartphones.

- **Software: Web browsers (Chrome, Firefox)**

Every browser uses a slightly different engine to run code. For Chrome, it uses the chromium engine and for Firefox uses the Gecko engine. Testing across this browser ensures compatibility, meaning a button that doesn't work on chrome doesn't fail to click in Firefox.

- **Connectivity: Internet Connection**

Since the system will be accessed online, the test needs to verify the following:

- ❖ API (Application Programming Interface) performance- how fast the system pull data from the sever
- ❖ Stability- test how the system handles slow connection or momentary drop in signal.
- ❖ Bandwidth (speed): this measure how much data can be sent at once. This testing goal is to make handle uploading large document from our teachers and high-resolution images for the archive without timing out.
- ❖ Mobile data usage-we will be testing how the system handles moving between signal strength.

9.RESOURCES

These are all the tools, people and materials require to design, develop, test and maintain the Student Huh system. These resources ensure that the project is completed efficiently and successfully.

1 Human resources - This are people involved I the project with each specific role:

- **Project manager**- is the one who oversees the entire project, assign tasks and ensure deadlines are met.
- **Developers** (fronted and backend) - fronted developers design the user interface (what user see) and they backend is the developers that handle the server, database and system logic.
- **Testers QA Quality Assurance team –** these team focuses on preventing problems and finding errors in the system. They check if all the features work

properly, for example, login and registration, viewing student information such as test date and assignment and the navigation between the pages. They also ensure quality standard to make sure the system is reliable (does not crush), secure (protect user data), and user-friendly.

2 Hardware resources

- Laptops/computers – used for coding design and testing.
- Smartphones/Tables- used to test responsiveness and mobile capability.
- Server- resource used for database

3 software resources- These are the programs and tools used during development:

- **Code Editors/IDEs** (Visual Studio Code) - used to write and mages code.
- **Version Control Systems** (GitHub) - helps teams collaborate, track changes, and manage project versions
- **Web Browsers** (Google Chrome, Mozilla Firefox, Microsoft Edge) - Used for testing how the system works online.
- **Database Management Systems** (MySQL)
  Used to store and manage user data such as student information.

4 Network resources

These involve connectivity and online access - Internet Connection – required for accessing the system online, uploading/downloading code from GitHub and testing live features

5 Time resource

This is the critical resource in the project development, the project has to be completed from the stipulated amount of time. The time is allocated in:

- Planning – the initial stage used to gather and analyze requirements. Also define the system goals and scope. Create test plans and QA strategies

- Development – this is where the system is built. Time is allocated foe writing codes or creating the system, following standards and guidelines and for performing early testing (like unit testing)
- Testing – the core QA phase where time is used for running different types of test (functional, performance, security), Retesting after fixes and

Deployment- This is when the system is released to users, time is allocated for:

- Final system checks
- Installation and configuration
- Monitoring the system after release

Proper time management ensures the project is completed on schedule.

6 **financial Resource**

These are the cost involved in the project such as internet data cost hosting services and software subscriptions. Ensure all necessary tools and services are available.

10.QUALITY ASSURANCE (QA)

Quality assurance ensures the system meets high standards and works reliably. Is the process of making sure a system is developed correctly, works properly, and meets the required standards before it is released to users. It focuses on **preventing problems**, not just fixing them.

Setting standards for coding and design - Before development begins, clear standards are defined for code quality (clean, readable code), system performance (fast and responsive), and security (protecting user data). This ensures everyone on the team follows the same guidelines. For example, in STUDENT HUB SYSTEM, quality Assurance would ensure that:

- Students can log in without errors
- Pages load correctly on phones and laptops
- Data (like student records) is accurate and secure
- The system works on browsers like Chrome, and Firefox

Identifying and reporting defects - If something doesn't meet the expectation standards, its document as a defect (bud, issue, or fault), along with step reproduction it and there is also **fixing and re-testing**. Here the developers or producers fix the issues and re-test to conform the problem is resolved and hasn't created a new one. There is also **Process Improvement,** QA isn't just about the product but it also evaluates the process. Team look for patterns in the defect and improve workflows to prevent future issues.

11.CHANGES (MAINTAINANCE AND UPDATES)

Changes refer to all modifications made to the system after testing and even after it has been released to users. These changes are part of **system maintenance**, which ensures the Student Hub system continues to function effectively, remains up-to-date, and meets user needs over time. Here we fix Bug, after testing or real-world use, errors may still be discovered. Also improve features, the system based on the user feedback and experience:

- Adding new features enhance or enhance the existing ones
- Making the interface easier to use.
- Improving navigation design.

For example, adding notification system for student and improve dashboard layout.

Performance updates

The system is updated to improve speed, efficiency, or compatibility.

- Optimizing loading speed
- Ensuring the system works on new devices or browsers
- Updating technologies used in development

Version control and collaboration

Changes must be tracked and organized using tools like GitHub:

- Branches are created for new features or fixes – new features or fixes without affecting the main system.
- Changes are committed with clear messages – are saved work with clear message of what was done
- A pull request is made for review – is when a developer ask the team to review their changes before adding to the main project, which is the Student Hub System without disturbing the already existing features.
- Code is merged only after approval- it happens after approval, combining the changes into the main system.